

面向对象程序设计C++ 第4次实验报告

一、判断题

第1题

题目：这段代码运行后，输出结果是什么？

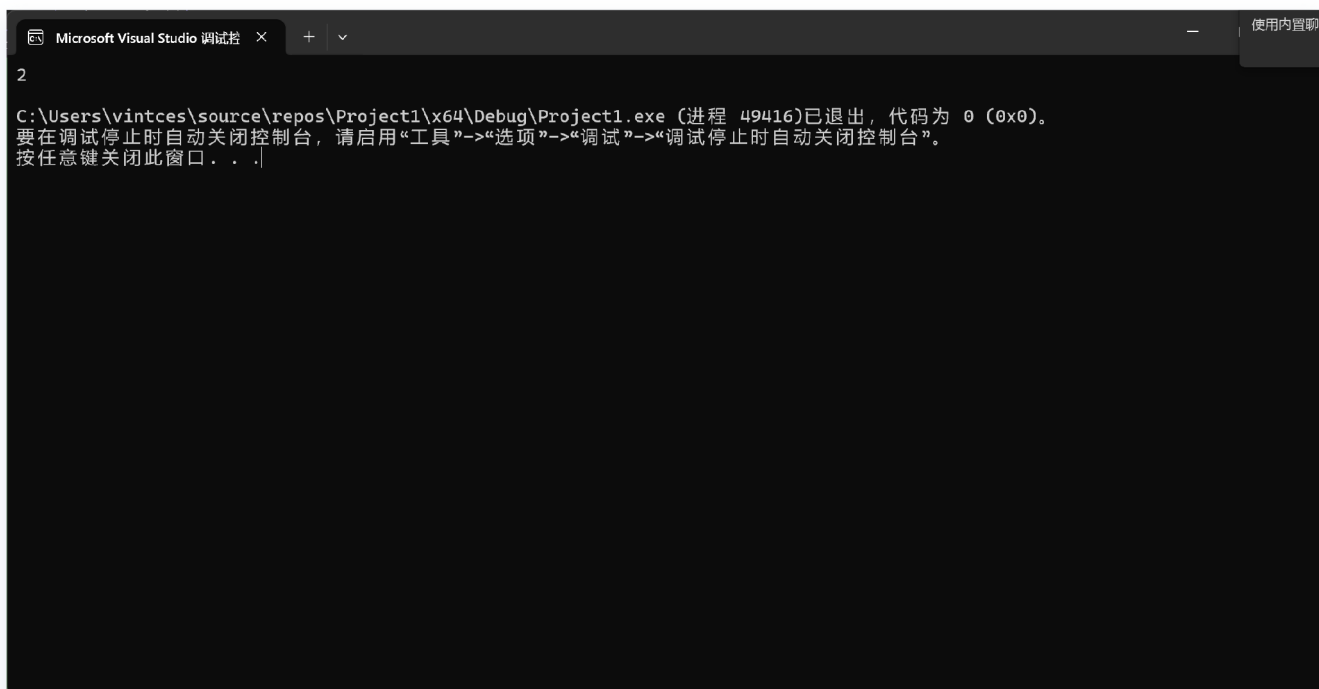
```
#include <iostream>
using namespace std;

class Counter {
private:
    int count;
public:
    void reset() {
        count = 0;
    }
    void increment() {
        count++;
    }
    int getCount() {
        return count;
    }
};

int main() {
    Counter c;
    c.reset();
    c.increment();
    c.increment();
    cout << c.getCount() << endl;
    return 0;
}
```

答案：C（输出 2）

运行结果：



分析：

程序中首先调用 `reset()` 函数，将对象 `c` 的成员变量 `count` 初始化为 0。随后连续调用两次 `increment()` 函数，每次调用都会使 `count` 的值增加 1，因此执行完成后 `count` 的值为 2。最后通过 `getCount()` 返回当前计数值并输出，所以程序的运行结果为 2。

第2题

题目：利用函数重载，以下代码的输出结果是什么？

```
#include <iostream>
#include <string>
using namespace std;

void logPrint(int i);
void logPrint(double d);
void logPrint(string s);

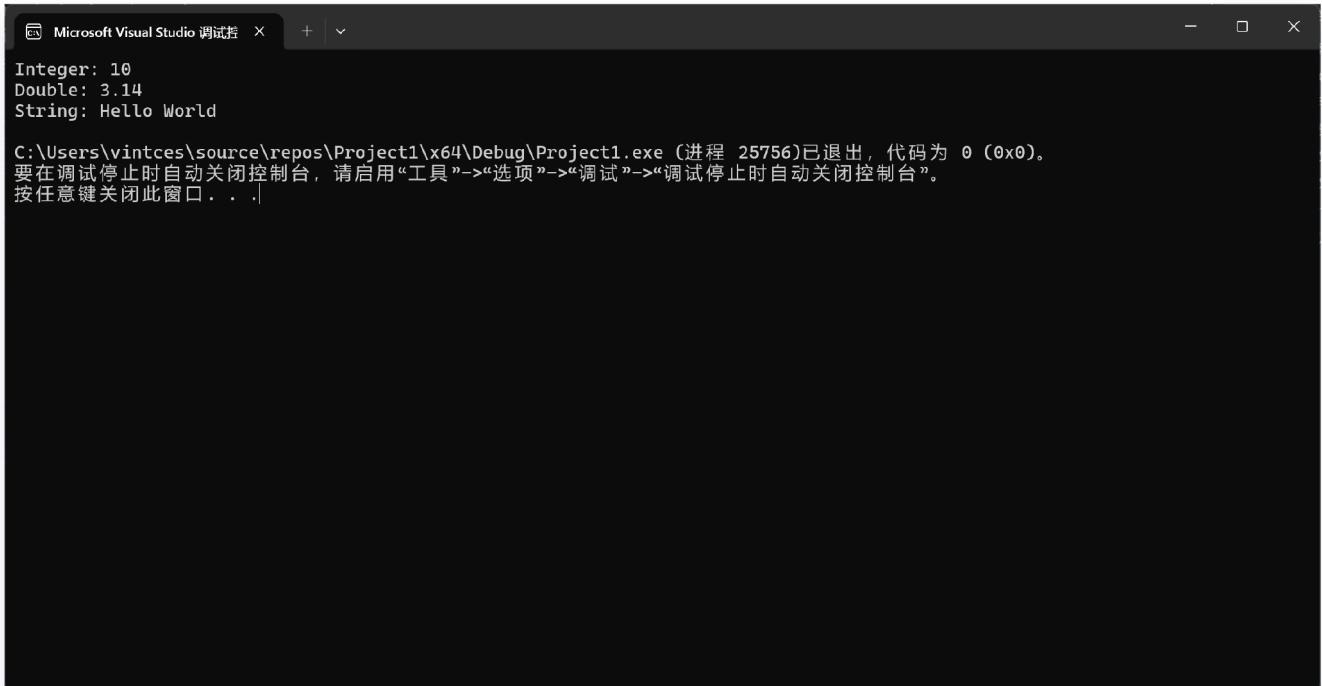
int main() {
    logPrint(10);
    logPrint(3.14);
    logPrint("Hello World");
    return 0;
}

void logPrint(int i) { cout << "Integer: " << i << endl; }
void logPrint(double d) { cout << "Double: " << d << endl; }
```

```
void logPrint(string s) { cout << "String: " << s << endl; }
```

答案：A

运行结果:



```
Microsoft Visual Studio 调试控制台
Integer: 10
Double: 3.14
String: Hello World

C:\Users\vintces\source\repos\Project1\x64\Debug\Project1.exe (进程 25756)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . |
```

分析:

该程序体现了函数重载根据实参类型选择对应函数的特点。调用 `logPrint(10)` 时, 实参为整型, 因此匹配 `logPrint(int)`; 调用 `logPrint(3.14)` 时, 实参为双精度浮点型, 因此匹配 `logPrint(double)`; 调用 `logPrint("Hello World")` 时, 字符串字面量可以转换为 `std::string` 类型, 因此匹配 `logPrint(string)`。三次调用分别输出整型、浮点型和字符串对应的内容。

第3题

题目：以下代码说法正确的是？

```
#include <iostream>
using namespace std;

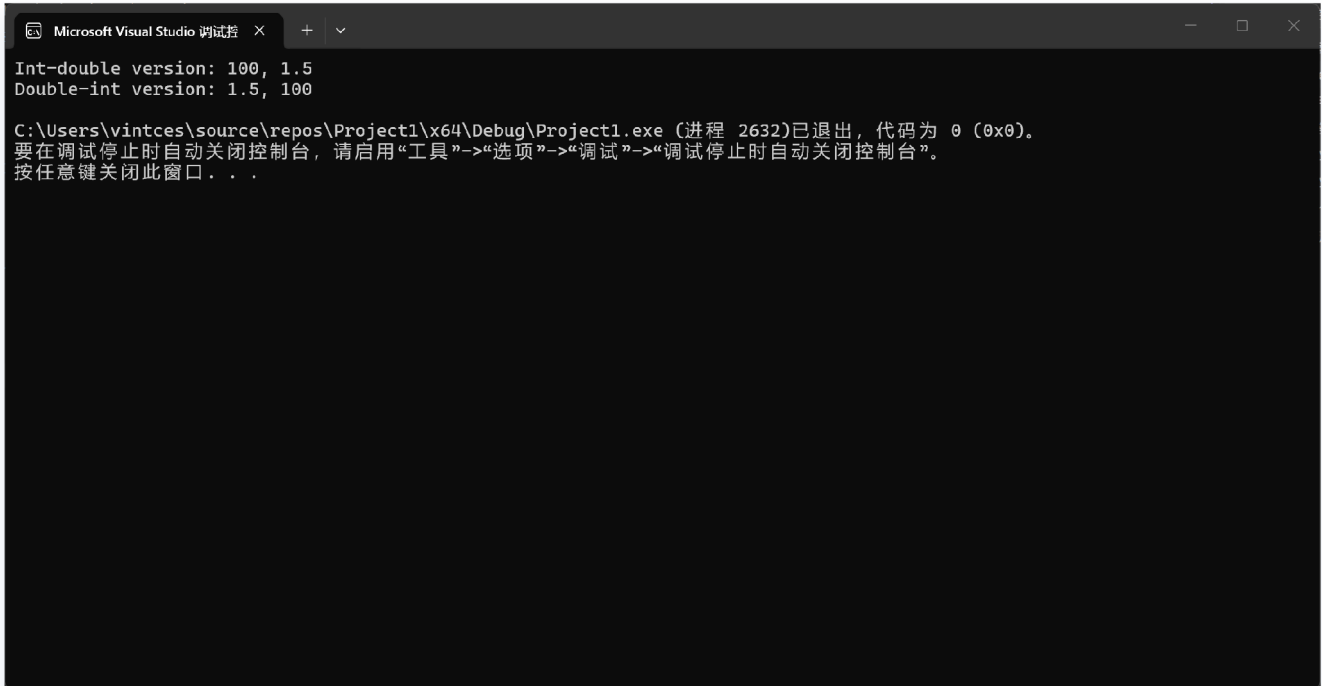
void process(int a, double b) {
    cout << "Int-double version: " << a << ", " << b << endl;
}

void process(double a, int b) {
    cout << "Double-int version: " << a << ", " << b << endl;
}
```

```
int main() {  
    process(100, 1.5); // Line H  
    process(1.5, 100); // Line I  
    return 0;  
}
```

答案：BD

运行结果：



```
Microsoft Visual Studio 调试器 x + v  
Int-double version: 100, 1.5  
Double-int version: 1.5, 100  
C:\Users\vintces\source\repos\Project1\x64\Debug\Project1.exe (进程 2632)已退出，代码为 0 (0x0)。  
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。  
按任意键关闭此窗口...
```

分析：

在本题中，两个 `process` 函数的参数类型相同，但参数排列顺序不同，分别为 `process(int, double)` 和 `process(double, int)`。在 C++ 中，函数重载不仅可以根据参数个数和参数类型进行区分，也可以根据参数类型的排列顺序进行区分，因此这两个函数构成合法的重载关系。

执行 `process(100, 1.5)` 时，第一个实参是 `int` 类型，第二个实参是 `double` 类型，能够与 `process(int, double)` 精确匹配；执行 `process(1.5, 100)` 时，第一个实参是 `double` 类型，第二个实参是 `int` 类型，能够与 `process(double, int)` 精确匹配。因此 Line H 和 Line I 均不存在二义性，也不会产生编译错误。综上，正确选项为 B、D。

第4题

题目：关于类的构造函数和析构函数，以下哪个描述是**错误的**？

答案：B ("一个类可以有多个析构函数"是错误的)

分析：

构造函数和析构函数都是类中非常重要的特殊成员函数。构造函数在对象创建时被调用，可以根据不同的参数列表进行重载，因此一个类中可以定义多个构造函数。析构函数则在对象生命周期结束时自动调用，主要用于释放对象占用的资源，例如动态分配的内存、文件句柄等。

需要注意的是，析构函数不能被重载。一个类中只能有一个析构函数，并且析构函数没有返回值，也不能带参数。因此，选项 B 中“一个类可以有多个析构函数”的说法不正确。

第5题

题目：在C++中使用 `new` 和 `delete` 操作符时，下列哪项描述是正确的？

答案：AD

分析：

`new` 和 `delete` 是 C++ 中进行动态内存管理的重要操作符。使用 `new` 在堆区申请内存后，程序员需要在合适的位置使用 `delete` 释放该内存，否则可能造成内存泄漏。因此，选项 A 是正确的。同时，`new` 和 `delete` 本身就是 C++ 用于管理动态分配内存的机制，所以选项 D 也正确。

对于数组类型的动态内存，应当使用 `new[]` 进行申请，并使用 `delete[]` 进行释放。如果使用 `new[]` 创建数组后只用 `delete` 释放，会导致未定义行为。除此之外，对并非由 `new` 申请得到的内存调用 `delete` 也不安全，同样属于未定义行为。因此，选项 B 和 C 的说法不正确。

二、编程题

2.1 题目描述

设计一个简单的图书管理系统，实现以下功能：

- **添加图书 (ADD)：**根据输入的图书信息，将图书添加到系统中。
- **删除图书 (DELETE)：**根据图书编号从系统中删除对应图书。
- **查询图书 (FIND)：**根据图书编号查询并显示图书的详细信息。
- **借阅图书 (BORROW)：**将图书标记为借出状态，并记录借阅者信息。
- **归还图书 (RETURN)：**将借出的图书标记为可借状态，并清除借阅者信息。

其中 `main.cpp`、`library.h`、`book.h` 文件已给，需要实现 `book.cpp` 和 `library.cpp`。

2.2 设计思路

整体结构

本系统由两个核心类构成：

Book 类 负责封装单本图书的全部信息与借阅状态。状态使用 `std::pair<bool, string>` 表示，其中 `bool` 为 `true` 表示可借（loanable），`false` 表示已借出（borrowed），`string` 存储当前借阅者姓名。提供两个重载的 `change_state()` 方法分别处理借出与归还两种状态切换。

Library 类 使用 `std::vector<Book>` 统一管理所有图书。所有涉及图书查找的操作（查询、删除、借阅、归还）均通过遍历 `vector` 按 ID 匹配目标图书，找不到时输出对应的失败提示。

关键设计点

1. 状态的二元表示：`state.first` 为 `true` 代表可借，`false` 代表已借出；借阅者姓名存于 `state.second`，归还时清空。
2. 重载 `change_state()`：无参版本用于归还，带 `string` 参数版本用于借出，接口清晰对称。
3. 允许借出状态下删除：`delete_book` 只按 ID 查找，不判断借阅状态，符合题目“借出后可正常删除”的约定。
4. 借阅与归还的状态前置校验：`borrow_book` 需确认图书当前为可借状态，`return_book` 需确认图书当前为借出状态，否则输出失败。

2.3 关键代码

book.cpp

```
#include "book.h"

Book::Book(int ID, string name, string author, string publishing_house)
    : ID(ID),
      name(name),
      author(author),
      publishing_house(publishing_house),
      state(true, "") // 初始状态: 可借, 无借阅者
{
}

int Book::get_ID() { return ID; }
string Book::get_name() { return name; }
string Book::get_author() { return author; }
string Book::get_publishing_house() { return publishing_house; }
```

```

bool    Book::get_state()                { return state.first; }

// 归还：恢复可借状态，清空借阅者
void Book::change_state()
{
    state.first = true;
    state.second = "";
}

// 借出：设为不可借，记录借阅者姓名
void Book::change_state(string name)
{
    state.first = false;
    state.second = name;
}

void Book::print_message()
{
    cout << "ID:"                << ID                << endl;
    cout << "name:"              << name              << endl;
    cout << "author:"            << author            << endl;
    cout << "publishing_house:" << publishing_house << endl;
    if (state.first) {
        cout << "state:loanable" << endl;
    } else {
        cout << "borrower:"      << state.second << endl;
        cout << "state:borrowed" << endl;
    }
}

```

library.cpp

```

#include "library.h"

void Library::add_book(Book book)
{
    Books.push_back(book);
    cout << "Added:" << book.get_name() << endl;
}

void Library::delete_book(int book_ID)
{
    for (auto it = Books.begin(); it != Books.end(); ++it) {

```

```

        if (it->get_ID() == book_ID) {
            cout << "Deleted:" << it->get_name() << endl;
            Books.erase(it);
            return;
        }
    }
    cout << "Delete failed" << endl;
}

void Library::find_book(int book_ID)
{
    for (auto &book : Books) {
        if (book.get_ID() == book_ID) {
            book.print_message();
            return;
        }
    }
    cout << "Find Failed" << endl;
}

void Library::borrow_book(int book_ID, string borrower)
{
    for (auto &book : Books) {
        if (book.get_ID() == book_ID && book.get_state()) {
            book.change_state(borrower);
            cout << "Borrowed:" << book.get_name() << endl;
            return;
        }
    }
    cout << "Borrow Failed" << endl;
}

void Library::return_book(int book_ID)
{
    for (auto &book : Books) {
        if (book.get_ID() == book_ID && !book.get_state()) {
            book.change_state();
            cout << "Returned:" << book.get_name() << endl;
            return;
        }
    }
    cout << "Return Failed" << endl;
}

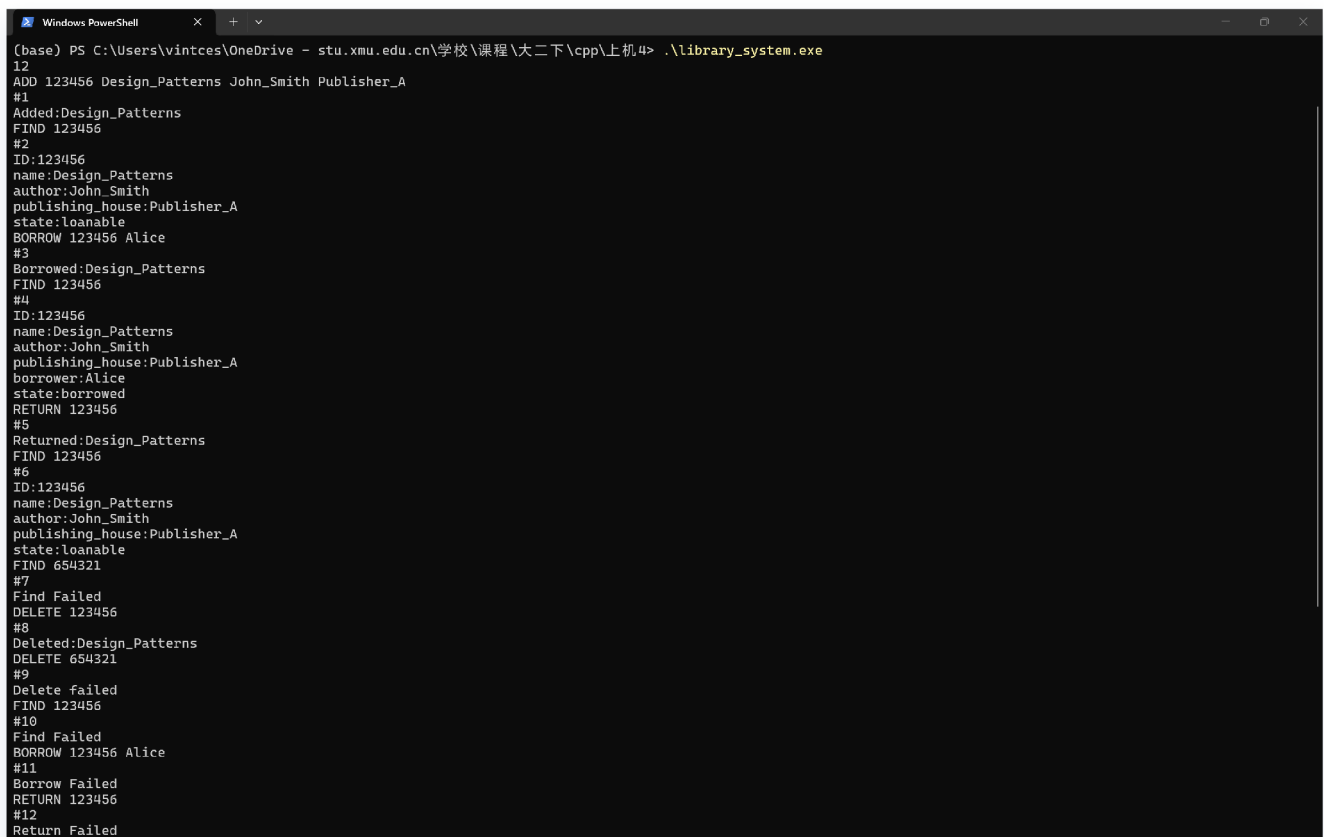
```


2.4 程序运行结果

测试输入：

```
12
ADD 123456 Design_Patterns John_Smith Publisher_A
FIND 123456
BORROW 123456 Alice
FIND 123456
RETURN 123456
FIND 123456
FIND 654321
DELETE 123456
DELETE 654321
FIND 123456
BORROW 123456 Alice
RETURN 123456
```

实际输出：



```
(base) PS C:\Users\vintces\OneDrive - stu.xmu.edu.cn\学校\课程\大二下\cpp\上机4> .\library_system.exe
12
ADD 123456 Design_Patterns John_Smith Publisher_A
#1
Added:Design_Patterns
FIND 123456
#2
ID:123456
name:Design_Patterns
author:John_Smith
publishing_house:Publisher_A
state:loanable
BORROW 123456 Alice
#3
Borrowed:Design_Patterns
FIND 123456
#4
ID:123456
name:Design_Patterns
author:John_Smith
publishing_house:Publisher_A
borrower:Alice
state:borrowed
RETURN 123456
#5
Returned:Design_Patterns
FIND 123456
#6
ID:123456
name:Design_Patterns
author:John_Smith
publishing_house:Publisher_A
state:loanable
FIND 654321
#7
Find Failed
DELETE 123456
#8
Deleted:Design_Patterns
DELETE 654321
#9
Delete failed
FIND 123456
#10
Find Failed
BORROW 123456 Alice
#11
Borrow Failed
RETURN 123456
#12
Return Failed
```

2.5 结果分析

编号	操作	输出	说明
#1	ADD 123456	Added:Design_Patterns	添加成功
#2	FIND 123456	完整信息, state:loanable	新书默认可借
#3	BORROW 123456 Alice	Borrowed:Design_Patterns	借出成功
#4	FIND 123456	含 borrower:Alice, state:borrowed	借出状态正确记录
#5	RETURN 123456	Returned:Design_Patterns	归还成功
#6	FIND 123456	state:loanable	状态正确恢复
#7	FIND 654321	Find Failed	图书不存在
#8	DELETE 123456	Deleted:Design_Patterns	删除成功
#9	DELETE 654321	Delete failed	图书不存在
#10	FIND 123456	Find Failed	已被删除
#11	BORROW 123456 Alice	Borrow Failed	图书不存在
#12	RETURN 123456	Return Failed	图书不存在

实际输出与预期输出完全一致，程序逻辑正确。